

DESIGN PATTERNS · CREATIONAL

Abstract Factory

Creating families of related objects without naming their concrete classes



What is Abstract Factory?

A creational pattern that lets you produce families of related objects without specifying their concrete classes.

Creational pattern

Families, not single objects

Client sees only interfaces

02 · THE PROBLEM

A furniture shop simulator

Products come in a **family**: Chair + Sofa + Coffee Table.

Each family comes in **variants**:
Modern, Victorian, Art Deco.

Every order must stay inside one variant.

Chair	Sofa	Coffee Table	Modern
Modern Chair	Modern Sofa	Modern CoffeeTable	Victorian
Victorian Chair	Victorian Sofa	Victorian CoffeeTable	Art Deco
ArtDeco Chair	ArtDeco Sofa	ArtDeco CoffeeTable	

Two requirements



Objects must match

A Modern sofa beside Victorian chairs is a defect. Customers expect every piece in an order to match.



Core code must stay stable

Vendors update their catalogues often. Adding a product or a variant should not mean editing existing code.

Goal: create objects that always belong to one family, without hard-coding their classes.

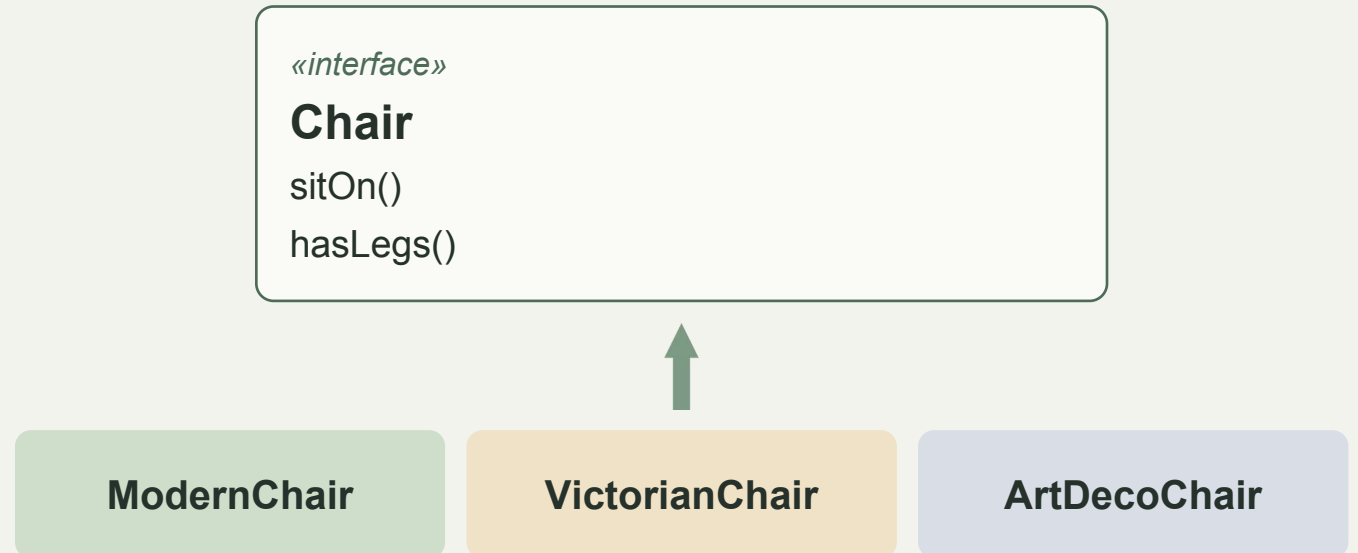
03 · THE SOLUTION

Step 1: One interface per product

Declare an interface for each distinct product: **Chair**, **Sofa**, **CoffeeTable**.

Every variant implements that interface.

The same hierarchy is built for Sofa and CoffeeTable.

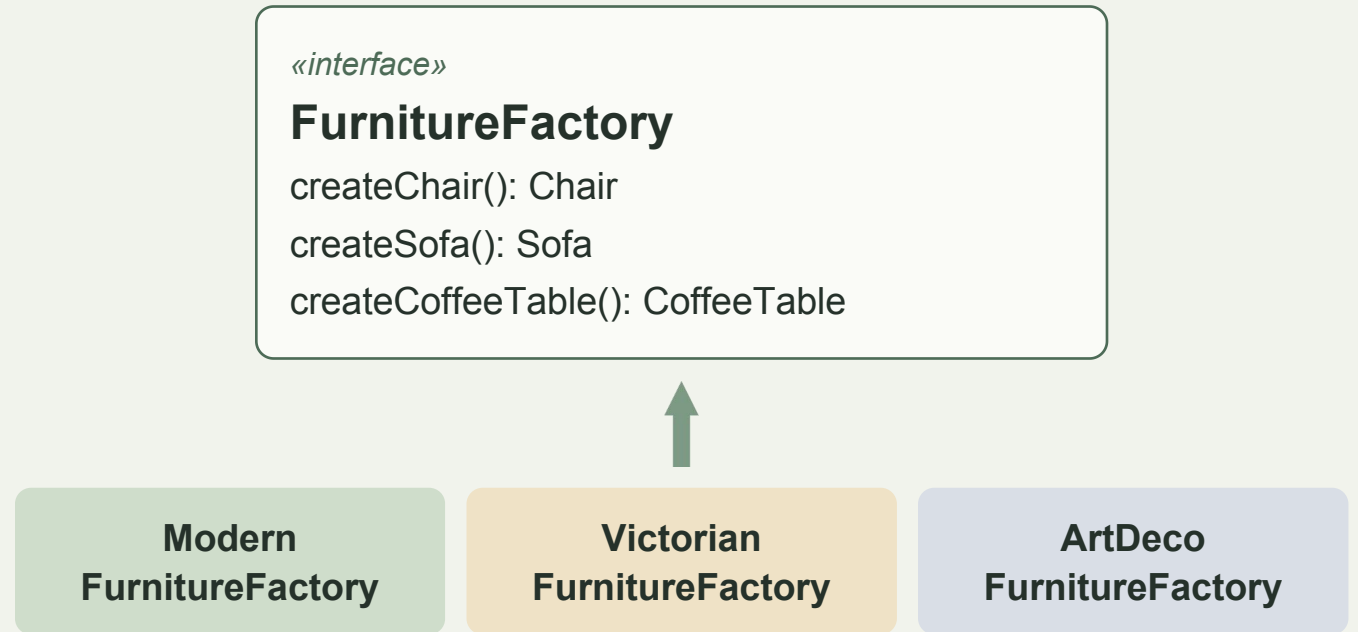


Step 2: An abstract factory interface

Declare one creation method for every product in the family.

Each method returns the **abstract** product type.

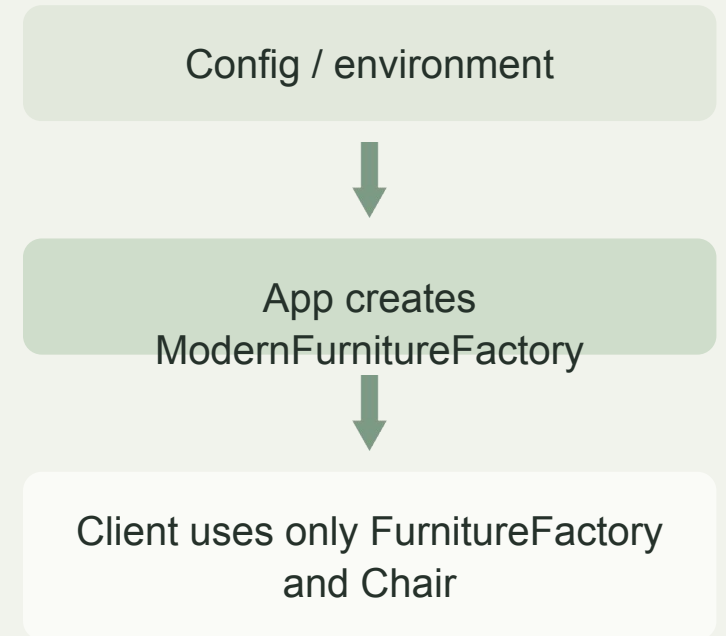
Write one concrete factory per variant.



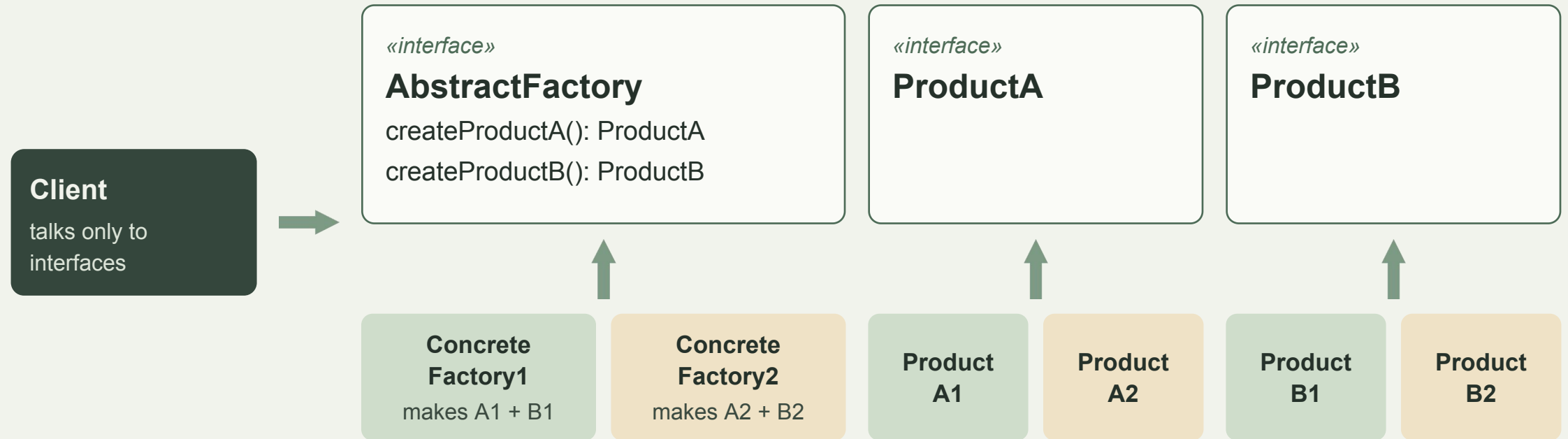
Step 3: The client stays abstract

The client never names a concrete class.

- ✓ It asks the factory for a chair and gets a **Chair**. Modern or Victorian, it treats them the same.
- ✓ Products from one factory always match each other.
- ⚙ The app picks the concrete factory once, at start-up, from configuration or environment.



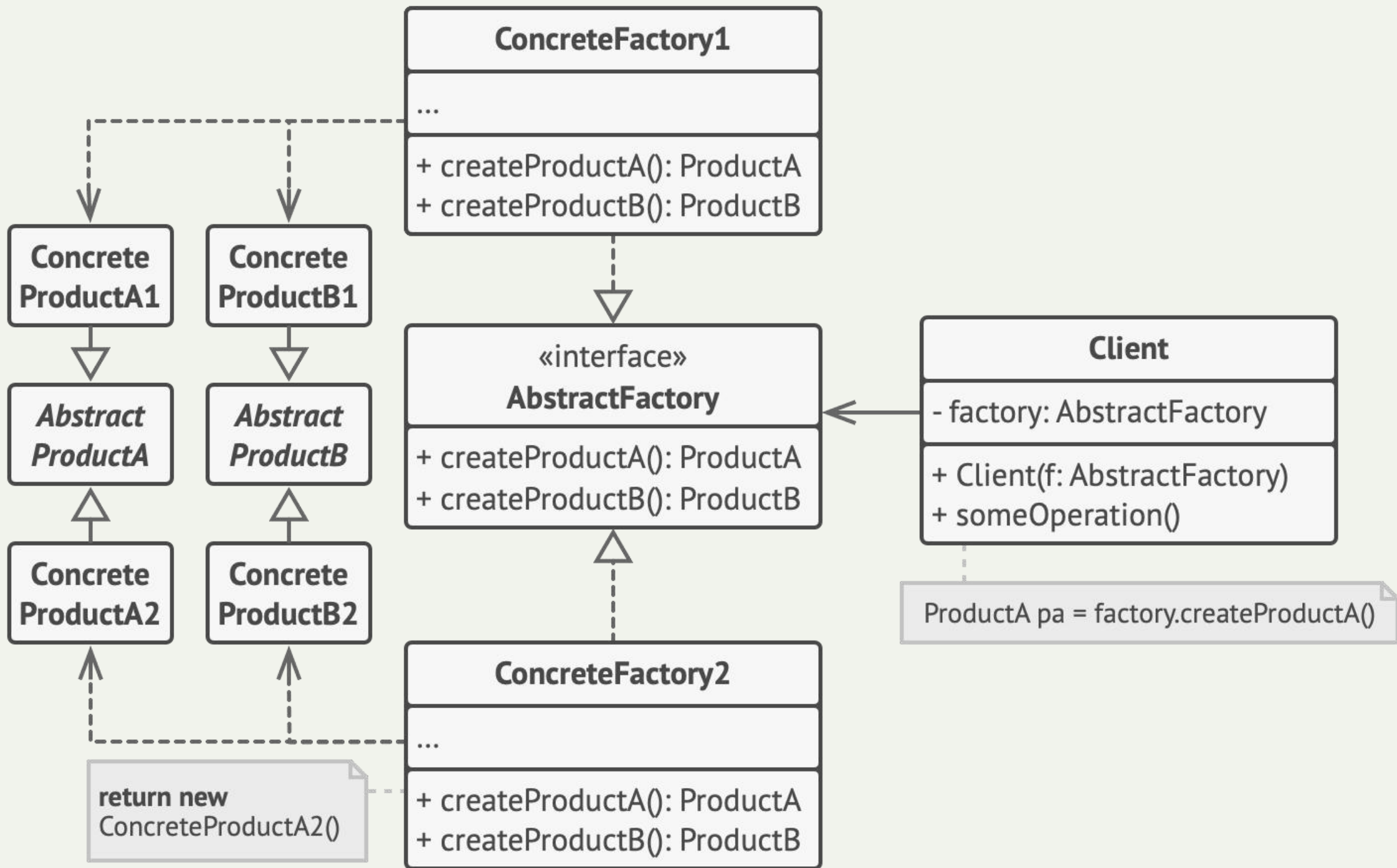
Structure at a glance



Arrows ↑ mean "implements". Colour marks the variant: green = variant 1, sand = variant 2.

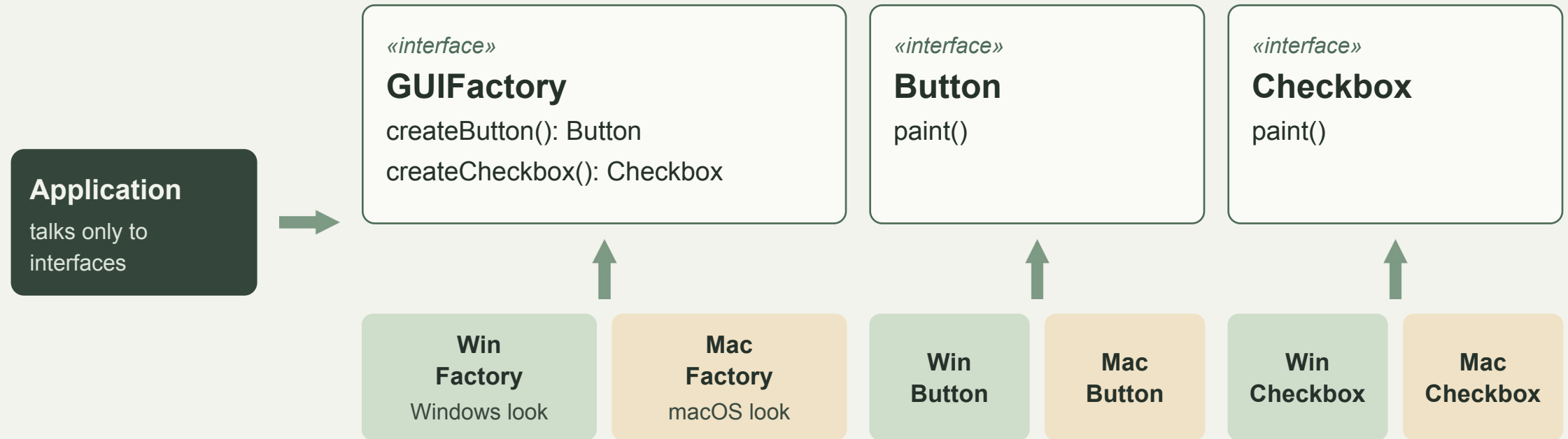
The five participants

- 1 **Abstract Products:** interfaces for the distinct products that make up a family.
- 2 **Concrete Products:** implementations of each abstract product, grouped by variant.
- 3 **Abstract Factory:** declares one creation method per abstract product.
- 4 **Concrete Factories:** one per variant; each creates only that variant's products.
- 5 **Client:** works with any factory or product through the abstract interfaces.



05 · EXAMPLE

Cross-platform UI elements



At launch the app checks the OS, creates WinFactory or MacFactory, and hands it to Application.

Pseudocode: the factories

```
interface GUIFactory
  createButton(): Button
  createCheckbox(): Checkbox

class WinFactory implements GUIFactory
  createButton(): Button
    return new WinButton()
  createCheckbox(): Checkbox
    return new WinCheckbox()

class MacFactory implements GUIFactory
  // same methods, returning
  // MacButton and MacCheckbox
```

What to notice

Signatures return abstract types.

Bodies create one concrete variant.

So WinFactory never returns a Mac checkbox.

Pseudocode: client and start-up

```
class Application
  constructor(factory: GUIFactory)
    this.factory = factory
  createUI()
    button = factory.createButton()
  paint()
    button.paint()

// at start-up: pick the factory once
config = readApplicationConfig()
if (config.OS == "Windows") factory = new WinFactory()
else if (config.OS == "Mac") factory = new MacFactory()
app = new Application(factory)
```

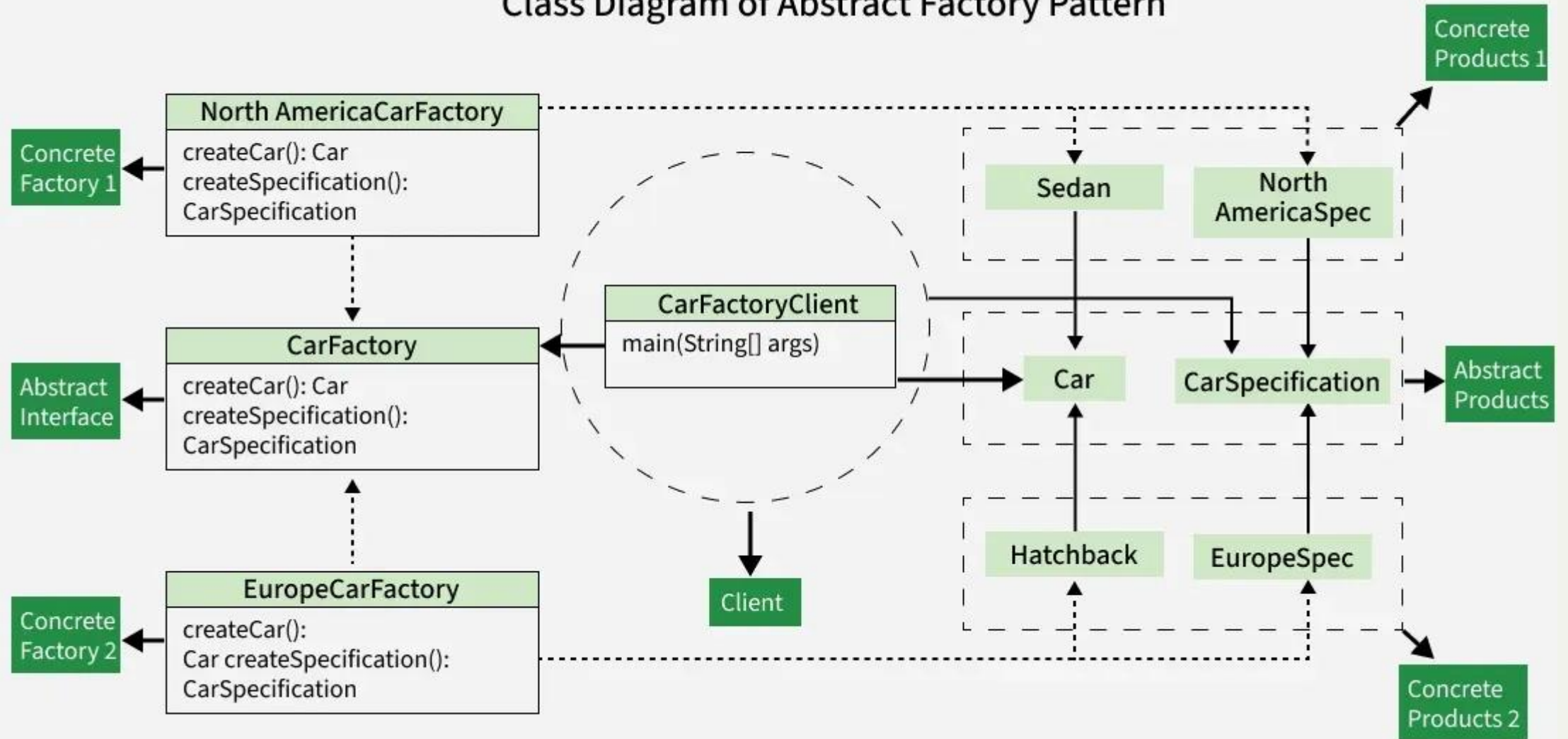
Why it works

Application knows only GUIFactory, Button and Checkbox.

The factory is chosen once, at start-up.

New OS? Add a factory and products; the client is untouched.

Class Diagram of Abstract Factory Pattern



A car maker selling in two regions

North America

Car: Sedan

Specification: safety features that meet local regulations

Europe

Car: SUV

Specification: fuel efficiency and emissions that meet EU standards

A region's car and its specification must always ship together.

Version 1: the client calls new

```
String region = "Europe";
Car car;
CarSpecification spec;
if (region.equals("NorthAmerica")) {
    car = new Sedan();
    spec = new NorthAmericaSpecification();
} else {
    car = new SUV();
    spec = new EuropeSpecification();
}
car.assemble();
spec.display();
```

Three problems

The client names every concrete class.

Region checks repeat wherever objects are made.

Nothing stops Sedan + EuropeSpecification.

Version 2: one factory per product

```
class CarCreator {  
    static Car create(String region) {  
        return isNA(region) ? new Sedan() : new SUV();  
    }  
}  
class SpecificationCreator {  
    static CarSpecification create(String region) {  
        return isNA(region)  
            ? new NorthAmericaSpecification()  
            : new EuropeSpecification();  
    }  
}
```

```
Car car = CarCreator.create("NorthAmerica");  
var spec = SpecificationCreator.create("Europe");  
// compiles, but the pair is mismatched!
```

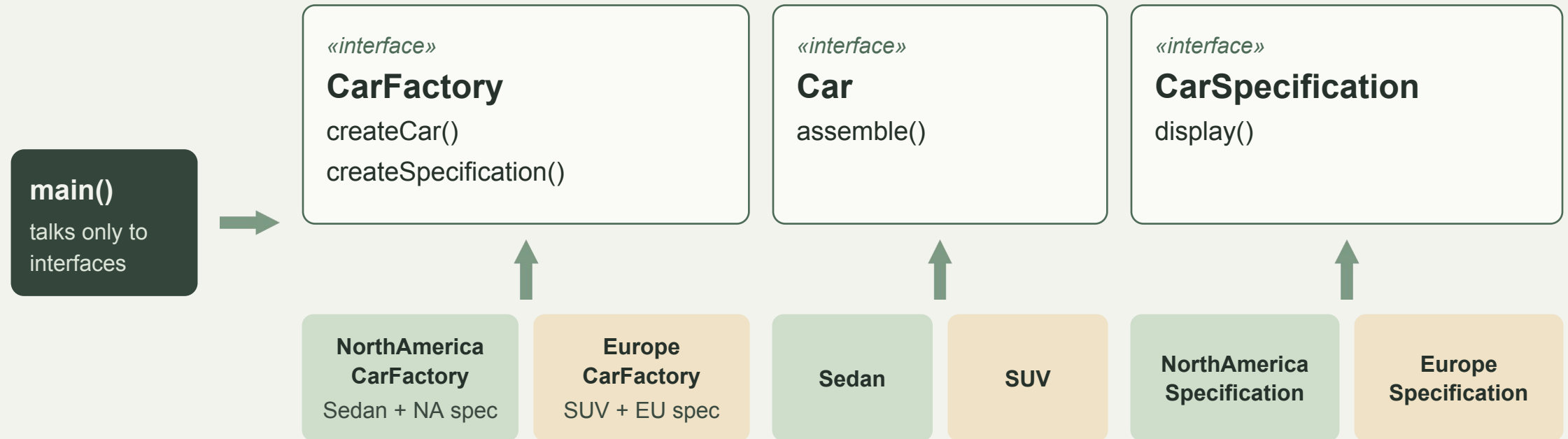
Better, but...

The client now uses only interfaces.

The two factories are independent, so they can disagree.

A new region means editing every factory.

Version 3: Abstract Factory



Arrows ↑ mean “implements”. Green = North America family, sand = Europe family.

Abstract Factory in the Java code

```
interface CarFactory {  
    Car createCar();  
    CarSpecification createSpecification();  
}  
class NorthAmericaCarFactory implements CarFactory {  
    Car createCar() { return new Sedan(); }  
    CarSpecification createSpecification() {  
        return new NorthAmericaSpecification(); }  
}  
// client: the only line naming a concrete class  
CarFactory f = new NorthAmericaCarFactory();  
f.createCar().assemble();  
f.createSpecification().display();
```

What to notice

Both methods return abstract types.

One factory object, so car and spec always match.

EuropeCarFactory returns SUV + EuropeSpecification.

Three versions side by side

	Naive new	Factory	Abstract Factory
Client names concrete classes	Yes	No	Only to pick the factory
Car + spec always match	No	No	Yes
Adding a region	Edit every if/else	Edit every factory	Add one factory class

What we gain, and the cost

Benefits

- ✓ **Consistency:** a region's car and spec never mix.
- ✓ **Loose coupling:** main sees only three interfaces.
- ✓ **New region:** add one class, e.g. AsiaCarFactory.
- ✓ **Single Responsibility:** one class per region.

Trade-off



A new product type, such as Warranty, means changing CarFactory and every concrete factory.

Code review: three small fixes

01

SUV prints “Hatchback”

SUV.assemble() should print
“Assembling SUV car.”

02

Two classes, one file

Move EuropeCarFactory into
its own
EuropeCarFactory.java.

03

Hard-coded factory choice

Pick the factory once from
config, then pass it to the
rest of the code.

When to use Abstract Factory



Families of related products

Your code must work with several families but should not depend on their concrete classes, because they are unknown up front or you want room to extend.



Factory methods crowd a class

A class with many factory methods blurs its main job. Extract them into a stand-alone factory or a full Abstract Factory.

How to implement: six steps

01

Map product types against their variants in a matrix.

02

Give each product type an interface; variants implement it.

03

Declare the abstract factory: one create method per product.

04

Write one concrete factory class per variant.

05

At start-up, pick a factory from config and pass it on.

06

Replace direct constructor calls with factory calls.

Benefits and trade-offs

Benefits

- ✓ Products from one factory are guaranteed compatible.
- ✓ Client code is decoupled from concrete products.
- ✓ **Single Responsibility:** creation code lives in one place.
- ✓ **Open/Closed:** add variants without breaking clients.

Trade-off



Many new interfaces and classes. The code can become more complex than the problem needs.

FACTORY METHOD VS ABSTRACT FACTORY

The key question: one product or a family?



Factory Method

Creates **one product**. A subclass overrides a single method to decide which class to create.



Abstract Factory

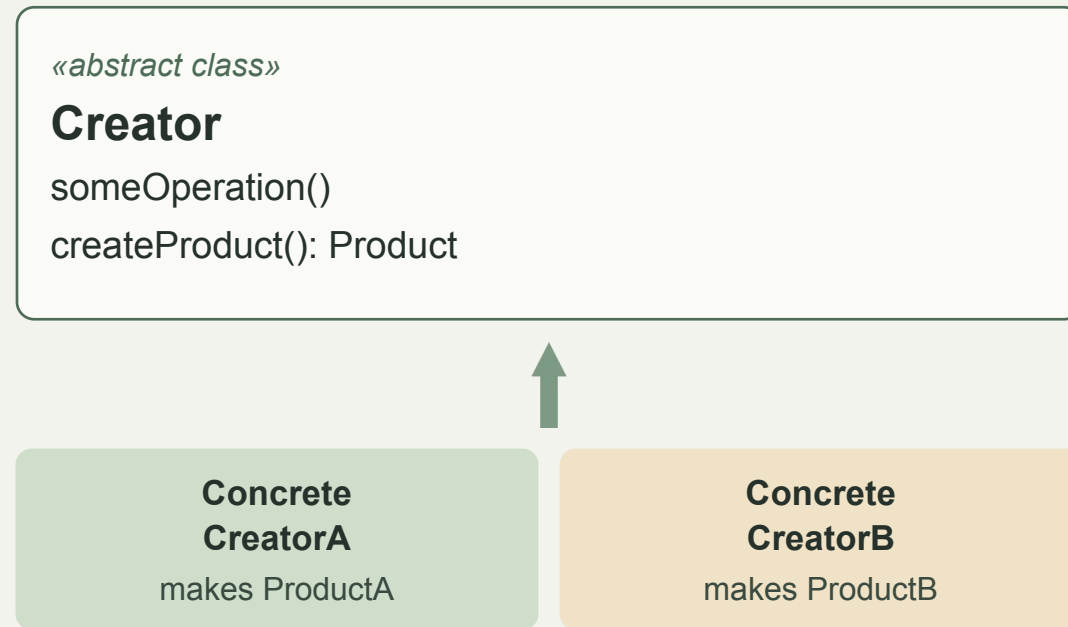
Creates a **family of related products**. A factory object decides the variant of every product at once.

“Factory pattern” here means Factory Method (GoF). A *Simple Factory* is just a static method with a conditional, as in Version 2 of the car case study.

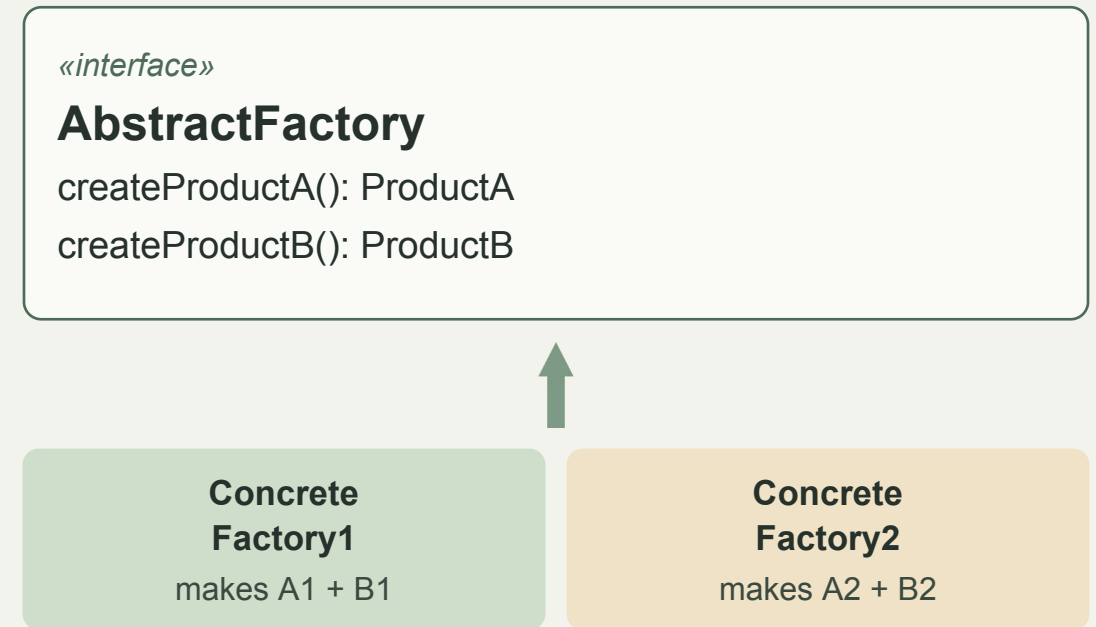
FACTORY METHOD VS ABSTRACT FACTORY

Structure side by side

Factory Method: one method



Abstract Factory: a set of methods



FACTORY METHOD VS ABSTRACT FACTORY

Point-by-point comparison

Aspect	Factory Method	Abstract Factory
Creates	One product	A family of products
How	Subclass overrides a method	Client is given a factory
Creation methods	One	One per product type
Consistency	Not addressed	Products always match
Easy to add	New product + new creator	A new family (one factory)
Hard to add	Rarely an issue	A new product type
Complexity	Lower	Higher: more interfaces

FACTORY METHOD VS ABSTRACT FACTORY

The same car domain, two patterns

Factory Method

```
abstract class Dealer {
    abstract Car createCar();
    void deliver() {
        createCar().assemble();
    }
}
class NorthAmericaDealer extends Dealer {
    Car createCar() {
        return new Sedan();
    }
}
// subclass decides: ONE product
```

Abstract Factory

```
interface CarFactory {
    Car createCar();
    CarSpecification createSpecification();
}
class NorthAmericaCarFactory implements CarFactory {
    Car createCar() { return new Sedan(); }
    CarSpecification createSpecification() {
        return new NorthAmericaSpecification();
    }
}
// object decides: a FAMILY of products
```

Which one should you use?



Use Factory Method when...

You create one kind of product and want subclasses to choose its concrete class.



Use Abstract Factory when...

Several products are created together and must always match each other.

They combine: an abstract factory's create methods are often factory methods. Many designs start with Factory Method and grow into Abstract Factory.

How it relates to other patterns

Pattern	Relationship to Abstract Factory
Factory Method	Designs often start with it, then evolve into Abstract Factory.
Builder	Builds step by step; Abstract Factory returns products at once.
Prototype	Can implement the creation methods of an abstract factory.
Facade	Alternative when you only need to hide object creation.
Bridge	The factory hides which implementations an abstraction needs.
Singleton	Concrete factories are often implemented as singletons.

KEY TAKEAWAYS

One factory, one consistent family

01

Group related products into families and give each product an interface.

02

One concrete factory per variant guarantees that products match.

03

Clients depend only on abstractions, so new variants plug in without edits.

Questions?